



INSTITUTO POLITÉCNICO
NACIONAL



UNIDAD PROFESIONAL
INTERDISCIPLINARIA EN INGENIERÍA
Y TECNOLOGÍAS AVANZADAS

MULTIMEDIA

PRÁCTICA: FILTROS

ALUMNO: HERNÁNDEZ AVILÉS LUIS ALBERTO

BOLETA: 2023640387

GRUPO: 3TM2

PROFESOR: SIERRA ROMERO NOE

Vía WhatsApp se compartieron dos códigos en lenguaje Python, los cuales debían de compilarse en el entorno de Google Colab. Esto con la finalidad de poder visualizar los resultados luego de aplicar un filtro que convertía ciertas características de la imagen example001 y volcán respectivamente.

El código “**filterOne.py**” funge como un procesador de imágenes a nivel de bits muy rudimentario pero efectivo. Su función principal es aplicar un filtro de binarización (convertir a blanco y negro puro) a una imagen de formato BMP de 24 bits. ¿Cómo lo hace?, bueno pues el código decide si un píxel debe ser blanco o negro basándose en un punto medio numérico.

- Calcula un valor central ($16,777,215 / 2 = 8,388,607$).
- Lectura de 3 bytes: En un BMP de 24 bits, cada píxel se compone de 3 bytes (uno para Azul, uno para Verde y uno para Rojo). El código lee estos 3 bytes juntos y los convierte en un solo número entero grande (valor_int).

Finalmente, en el proceso de toma de decisión el código evalúa cada pixel por medio de una condición lógica:

- Si el valor es "bajo" (oscuro): Lo fuerza a ser Blanco [0xff, 0xff, 0xff].
- Si el valor es "alto" (claro): Lo fuerza a ser Negro [0x00, 0x00, 0x00].

Código:

```
from google.colab import files
import io
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from PIL import Image

# 1. Solicitar la subida
print("Por favor, sube tu imagen .bmp:")
uploaded = files.upload()

for filename in uploaded.keys():
    # --- PROCESAMIENTO BINARIO (Tu lógica original) ---
    file_bytes = io.BytesIO(uploaded[filename])
    output_name = 'resultado_binario.bmp'

    with open(output_name, 'wb') as fileo:
        metadata = file_bytes.read(54)
        fileo.write(metadata)

        blanco = [0xff, 0xff, 0xff]
        negro = [0x00, 0x00, 0x00]
        limite = (pow(2, 24) - 1) / 2

        while True:
            pixel_data = file_bytes.read(3)
            if len(pixel_data) == 3:
                valor_int = int.from_bytes(pixel_data, byteorder='little')
                # Mantenemos tu lógica: Oscuro -> Blanco / Claro -> Negro
                if valor_int < limite:
                    fileo.write(bytes(blanco))
                else:
```

```

        fileo.write(bytes(negro))
    else:
        break

# --- VISUALIZACIÓN ---
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

# Imagen Original
img_org = Image.open(io.BytesIO(uploaded[filename]))
axes[0].imshow(img_org)
axes[0].set_title(f"Original: {filename}")
axes[0].axis('off')

# Imagen Procesada
img_bin = Image.open(output_name)
axes[1].imshow(img_bin)
axes[1].set_title("Resultado (Binarización Inversa)")
axes[1].axis('off')

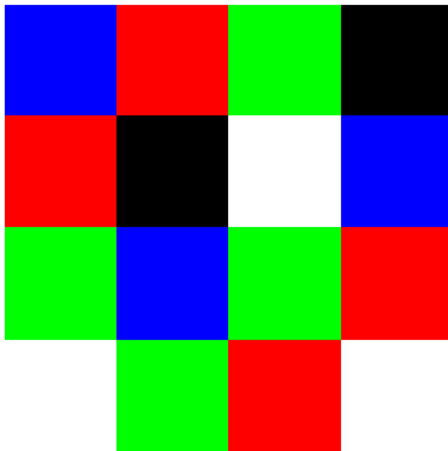
plt.show()

```

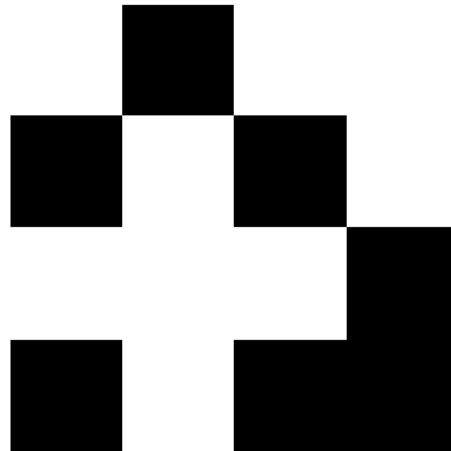
Resultados:

Por favor, sube tu imagen .bmp:
 example001.bmp
 *** **example001.bmp**(image/bmp) - 822 bytes, last modified: 10/2/2026 - 100% done
 Saving example001.bmp to example001 (1).bmp

Original: example001 (1).bmp



Resultado (Binarización Inversa)



Ahora analizando el segundo código proporcionado que fue "readBMP.py" el cual funge como una herramienta de inspección técnica para archivos BMP. A diferencia del anterior, este no modifica la imagen, sino que "abre las tripas" del archivo para mostrar exactamente qué información contiene byte por byte el archivo BMP en cuestión.

¿Cómo lo hace?

Verificación de la firma: Lee los dos primeros bytes del archivo. En un archivo BMP, estos siempre deben ser b'BM'. Es la forma en que el sistema sabe que realmente es un mapa de bits y no un PDF o un JPG renombrado. Por medio de (firm=file.read(2)).

Inspección del primer pixel: Salta directamente al final del encabezado (byte 54) para leer el primer color de la imagen. En el formato BMP de 24 bits, leerá 3 bytes que corresponden a los canales B (Blue), G (Green) y R (Red). Por medio de file.seek(54,0) seguido de pixel_first = file.read(3).

Volcado de datos de pixeles: El bloque while(True) recorre toda la imagen de 3 en 3 bytes y los imprime en la consola:

- Lectura masiva: Muestra en pantalla el valor hexadecimal de cada color de la imagen.
- Conteo: Al final, te dice cuántos píxeles procesó en total (No Pixels).

Código:

```
from google.colab import files
import io

# 1. Solicitar la subida del archivo manualmente
print("Por favor, sube la imagen (formato .bmp):")
uploaded = files.upload()

for filename in uploaded.keys():
    # Acceder a los datos del archivo subido en memoria
    file_bytes = io.BytesIO(uploaded[filename])

    print(f"\n--- Analizando: {filename} ---")

    # 2. Leer y mostrar la firma (Magic Number)
    firm = file_bytes.read(2)
    print(f"Firma del archivo: {firm}")

    # 3. Leer el primer píxel (saltando el encabezado de 54 bytes)
    file_bytes.seek(54, 0)
    pixel_first = file_bytes.read(3)
    print(f"Primer píxel (BGR): {pixel_first}")

    # 4. Volcar todos los píxeles y contarlos
    file_bytes.seek(54, 0) # Regresar al inicio de los datos de imagen
    no_pix = 0

    print("\nDatos de los píxeles (formato bytes):")
```

```
while True:
    pixel_data = file_bytes.read(3)
    if len(pixel_data) == 3:
        print(pixel_data)
        no_pix += 1
    else:
        # Si sobran bytes que no completan un píxel (padding) o termina el archivo
        break

print(f'\nTotal de Píxeles procesados: {no_pix}')
```

Resultados:

```
Por favor, sube la imagen (formato .bmp):
Elegir archivos example001.bmp
example001.bmp (image/bmp) - 822 bytes, last modified: 10/2/2026 - 100% done
Saving example001.bmp to example001.bmp

--- Analizando: example001.bmp ---
Firma del archivo: b'BM'
Primer píxel (BGR): b'\xff\xff\xff'

Datos de los píxeles (formato bytes):
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\x00\xff\x00'
b'\x00\xff\x00'
b'\x00\xff\x00'
b'\x00\xff\x00'
b'\x00\xff\x00'
b'\x00\xff\xff'
b'\x00\xff\xff'
b'\x00\xff\xff'
b'\x00\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\x00\xff\x00'
b'\x00\xff\x00'
b'\x00\xff\x00'
```

```
b'\x00\xff\x00'
b'\x00\x00\xff'
b'\x00\x00\xff'
b'\x00\x00\xff'
b'\x00\x00\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\x00\xff\x00'
b'\x00\xff\x00'
b'\x00\xff\x00'
b'\x00\xff\x00'
b'\x00\x00\xff'
b'\x00\x00\xff'
b'\x00\x00\xff'
b'\x00\x00\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\xff\xff\xff'
b'\x00\xff\x00'
b'\x00\xff\x00'
b'\x00\xff\x00'
b'\x00\xff\x00'
b'\x00\x00\xff'
b'\x00\x00\xff'
```

[illegible]

[¿Cómo hago para instalar bibliotecas de Python?](#)
[Carga datos de Google Drive](#)
[Muestra un ejemplo](#)

¿Qué puedo ayudarte a crear?

Gemini 2.5 Flash ▾ ▶

[illegible]

[¿Cómo hago para instalar bibliotecas de Python?](#)
[Carga datos de Google Drive](#)
[Muestra un ejemplo](#)

¿Qué puedo ayudarte a crear?

Gemini 2.5 Flash ▾ ▶

```

b'\x00\x00\xff+'
b'\x00\xff\x00'
...
b'\x00\xff\x00'
b'\x00\x00\x00'
b'\x00\x00\x00'
b'\x00\x00\x00'
b'\x00\x00\x00'
b'\x00\x00\x00'
b'\xff\x00\x00'
b'\xff\x00\x00'
b'\xff\x00\x00'
b'\x00\x00\xff'
b'\x00\x00\xff'
b'\x00\x00\xff'
b'\x00\xff\x00'
b'\x00\xff\x00'
b'\x00\x00\x00'
b'\x00\x00\x00'
b'\x00\x00\x00'
b'\x00\x00\x00'

```

Total de Píxeles procesados: 256